Introduction to Databases & E-R Model

Mr. S.K.Patil

Introduction to Database System

Introduction –

- **Data -**
  - Data is defined as **set of raw facts** about a entity that can be recorded.
  - Data is a **collection of facts and figures** that can be **processed to produce information**. It is **raw and unorganized**.
  - Data is represented in **various forms** like characters, text, numbers, images, audio, video, etc.
- **Information -**
  - Information is defined as **processed data that provides meaningful insights of data.**
  - Information is **organized** or classified data, which has some **meaningful values** for the receiver.

Introduction to Database System

Introduction –

- **Information -**
  - Information is data that has been converted into a more useful or intelligible form.
  - Information is the **processed data on which decisions and actions are based.**
  - Information has time and context associated with it.
  - Information may be **presented in tabular form or graphical form.**
- **Metadata -**
  - Metadata means **"data about data".** It is **data that describes other data**.
  - Metadata is as a **short explanation or summary of what the data is**.

Introduction to Database System

Introduction –

- **Database -**
  - The database is an **organized collection of structured data** to make it **easily accessible, manageable and update.**
  - The database is a **shared collection of logically related data** designed to meet the information needs of an organization.
  - A database in a place where the data is stored.
  - The database is a **single,** possibly **large repository of data** that can be **used simultaneously by many departments and users.**
  - A database is also defined as a **self-describing collection of integrated records.**
  - The database represents the entities, the attributes, and the logical relationships between the entities.

Introduction to Database System

Introduction –

- In today's world many **computing applications deal with large amounts of information** regularly.
- There is a **challenge** to **store the large amount of information**, **retrieve and manage this information in timely manner.**
- This can be **achieved by making use of services of Database Management System (DBMS).**
- With the help of DBMS, we can **insert, update and delete the data** stored in database.
- Also **collect the data**, **give a systematic representation** to it and also provides ways for the **data to be modified or extracted by users** or other programs.

Introduction to Database System

What is Database Management System? –

- A database management system (DBMS) is a **collection of interrelated data and a set of programs to access those data.**
- The collection of data referred as database which contains information relevant to an enterprise.
- **Primary goal of DBMS -**
  - **Store and retrieve** database **information** in **convenient and efficient manner.**
  - **Enables users** to **define, create, maintain, & control access to** the **database.**
  - **Allows users** to **define the database** through a **Data Definition Language(DDL).**
  - **Allows users** to **insert, update, delete, and retrieve data** from the database through a **Data Manipulation Language (DML).**
  - **Define structure of a document**
  - **Safety against system crash and unauthorized access**

Introduction to Database System

Database System Applications –

- Databases are widely used many application. Some of them are mentioned below –
  - **Banking** : For customer information, accounts, loans, and banking transactions.
  - **Universities** : For student information, course registrations, and grades.
  - **Credit card transactions** : For purchases on credit cards and generation of monthly statements.
  - **Finance**: For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds.
  - **Sales**: For customer, product, and purchase information.

Introduction to Database System

Database System Applications –

- Databases are widely used many application. Some of them are mentioned below –
  - **Airlines**: For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner − terminals situated around the world accessed the central database system through phone lines and other data networks.
  - **Telecommunication**: For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.

Introduction to Database System

Traditional File Processing System –

- Traditional file processing systems were introduced to **computerize the manual filing system.**
- File processing systems were developed to **store, manipulate and retrieve large files of data.**
- Traditional File Processing system is a **collection of application programs that perform services for the end-users**, such as insertion, deletion and updating existing files and adding new files to database etc.
- **Each program defines and manages its data.**

Introduction to Database System

Traditional File Processing System –

Introduction to Database System

Limitations of Traditional File Processing System –

- **Separation and isolation of data** –
  - In this approach each application has its own private files and users have little opportunity to share data outside their own applications.
  - Managers need to put more effort and time in extracting the information needed.

- **Duplication of data –**
  - Applications are often developed independently in file processing systems this results in unplanned duplicate data files.
  - Duplication of data results in wastage of storage space and data integrity problems.

Introduction to Database System

Limitations of Traditional File Processing System –

- **Data Independence -**
  - File descriptions are stored within each application program that access a given file, therefore any changes to a file structure requires changes to the file descriptions for all the programs that access the file.
  - It is difficult to locate all programs that are affected by such changes.
  - Sometimes errors are introduced while making changes.
- **Incompatible file formats -**
  - Different file formats are used by different application programs. Therefore one file format is not compatible to another application.

Introduction to Database System

Limitations of Traditional File Processing System –

- **Fixed Queries -**
  - The functionalities of file processing systems are fixed.
  - No scope to add new queries.

- **Large number of application programs –**
  - Each department has developed its own application programs.
  - Lack of awareness of existence of application programs led to development of similar kind of application programs by other departments.

Introduction to Database System

History of Database System –

- In early 1960s, the **first general purpose DBMS** was designed by Charles Bachman at General Electric, which was later, called as **IDS** (Integrated Data Store).
- In late 1960s, **IBM developed the IMS** (Information Management System) which was widely used.
- By the **joint venture of IBM and American Airlines**, the **SABRE system** was launched which help the people to reserve the tickets.
- The new data representation framework was initially launched by Edgar Codd, called the **Relational data model.**
- As the development of **relational DBMS** has reached to larger users and the number of benefits from the same, it was **widely accepted** and many corporate houses started using this system for their day to day activities.

Introduction to Database System

History of Database System –

- As the popularity of relational DBMS started increasing, soon **IBM**, in early 1980s, has **developed a SQL** (Structured Query Language) **for relational databases through** their **SYSTEM/R project.**
- Later in late 1980s, **SQL was standardized** and the version SQL-1999 was adopted by ANSI and ISO.
- Today we have multimedia databases, interactive video, streaming data, video libraries has completely change in the way in which data is stored which allowed the company to simplify their decision making process.

Introduction to Database System

Purpose of Database System –

Database Systems versus Traditional File based System

- Early days, database applications were built on top of file systems.
- Drawbacks of using file systems to store data:
  - **Data redundancy and inconsistency** - Multiple file formats, duplication of information in different files
  - **Difficulty in accessing data** - Need to write a new program to carry out each new task
  - **Data isolation** - multiple files and formats
  - **Integrity problems** - Integrity constraints become part of program code, Hard to add new constraints or change existing ones

Introduction to Database System

Purpose of Database System –

Database Systems versus Traditional File based System

- Drawbacks of using file systems to store data:
  - **Atomicity of updates** - Failures may leave database in an inconsistent state with partial updates carried out
  - e.g. transfer of funds from one account to another should either complete or not happen at all
  - **Concurrent access by multiple users –**
    - Concurrent access needed for performance
    - Uncontrolled concurrent accesses can lead to inconsistencies
    - e.g. two people reading a balance and updating it at the same time
  - **Security**

Introduction to Database System

Purpose of Database System –

Database Systems versus Traditional File based System

- Drawbacks of using file systems to store data:
  - **Database Approach** – All the above limitations of the file-based approach can be attributed to two factors:
    - The definition of the data is embedded in the application programs, rather than being stored separately and independently.
    - There is no control over the access and manipulation of data beyond that imposed by the application programs.
  - To become more effective, a new approach was required. Hence the database and the Database Management System (DBMS) were emerged.

Introduction to Database System

Advantages and Disadvantages of DBMS –

Advantages

- **Program Data Independence -**
  - The **separations of data descriptions** (metadata) **from** the **application programs** that use the data is called data independence.
  - In database approach, data descriptions are stored in central location called **repository**.
  - Changes in data definitions can be made without changing the application programs.
- **Minimal Data Redundancy –**
  - Data files are integrated into single logical structure called database. And data is stored in only one place in the database.
  - Database approach allows the designer to carefully **control the type and amount of redundancy.**

Introduction to Database System

Advantages and Disadvantages of DBMS –

Advantages

- **Improved Data Consistency -**
  - By **eliminating or controlling data redundancy** we can **overcome inconsistency in data.**
  - Updating data is simplified when data is stored in one place.
- **Improved Data Sharing –**
  - A **database** is designed as a **shared corporate resource**.
  - **Authorized users** are granted permission to **use the database** and each **user** is **provided one or more user views on data.**
  - A **user view** is **logical description of some portion** of the database that is **required by a user** to perform some task.

Introduction to Database System

Advantages and Disadvantages of DBMS –

Advantages

- **Improved Data Integrity -**
  - **Database integrity** refers to the **validity and consistency of stored data.**
  - **Integrity** is **expressed in terms of constraints**, which are **consistency rules** that the **database is not permitted to violate**.
  - **Constraints** may **apply to data items** within a **single record** or they may apply **to relationships between records**.
- **Improved Data Security -**
  - **Database security** is the **protection** of the database **from unauthorized users.**
  - Security **prevents unauthorized access** to data.

Introduction to Database System

Advantages and Disadvantages of DBMS –

Advantages

- **Enforcement of standards -**
  - **Integration allows** the Database Administrator **(DBA) to define and enforce the necessary standards.**
  - These may include **departmental, organizational, national, or international** standards.
- **Economy of scale –**
  - **Combining all** the **organization's operational data into one database,** and creating a set of applications that work on this one source of data, can result in **cost savings.**

Introduction to Database System

Advantages and Disadvantages of DBMS –

Advantages

- **Improved data accessibility and responsiveness -**
  - As a **result of integration**, **data** that crosses departmental boundaries is directly **accessible to the end-users**.
  - This provides a **system** with potentially much **more functionality**.
- **Increased productivity –**
  - **DBMS provides** many of the standard functions that the programmer would normally have to write in a **file-based application**.
  - At a basic level, the **DBMS provides all the low-level file handling routines** that are typical in application programs.

Introduction to Database System

Advantages and Disadvantages of DBMS –

Advantages

- **Improved backup and recovery services -**
  - Modern DBMSs provide facilities to **minimize the amount of processing** that is lost following a failure.
  - DBMS software **provides rich set of backup and recovery services**.
  - **Cold backup or hot backup** facilities are available.

Introduction to Database System

Advantages and Disadvantages of DBMS –

Disadvantages

- **Complexity -**
  - The provision of the functionality we expect of a good DBMS makes the DBMS an extremely **complex piece of software.**
  - Database designers and developers, the data and database administrators, and end-users must understand this functionality to take full advantage of it.
  - **Failure to understand the system** can **lead to bad design decisions**, which can have serious consequences for an organization.
- **Size -**
  - The **complexity and breadth of functionality makes** the **DBMS** an extremely **large piece of software**, occupying many megabytes of disk space and requiring substantial amounts of memory to run efficiently.

Introduction to Database System

Advantages and Disadvantages of DBMS –

Disadvantages

- **Cost of DBMSs -**
  - The **cost of DBMSs varies significantly**, depending on the environment and functionality provided.
  - A single-user DBMS for a personal computer may only cost US$100.
- **Additional hardware costs -**
  - The **disk storage requirements** for the DBMS and the database may necessitate the purchase of additional storage space.
  - To **achieve the required performance**, it may be **necessary to purchase a larger machine.**

Introduction to Database System

Advantages and Disadvantages of DBMS –

Disadvantages

- **Cost of conversion -**
  - In some situations, the cost of the DBMS and extra hardware may be insignificant compared with the cost of converting existing applications to run on the new DBMS and hardware.
  - This cost also includes the cost of training staff to use these new systems.
- **Higher impact of a failure -**
  - The **centralization of resources increases** the **vulnerability** of the system.
  - Since all users and applications rely on the availability of the DBMS, the **failure of certain components** can **bring operations to a halt.**

Introduction to Database System

Advantages and Disadvantages of DBMS –

Disadvantages

- **Performance -**
  - A file-based system is written for a specific application, such as invoicing. As a result, performance is generally very good.
  - However, the DBMS is written to be more general, to cater for many applications rather than just one. The effect is that **some applications may not run as fast as they used to.**

Introduction to Database System

View of Data –

- **View of data** describes **how the data is visualized at each level** of data abstraction.
- **Data Abstraction** allow developers to **keep complex data structures away from the users.**
- The **developers achieve** this by **hiding the complex data structures through levels of abstraction.**
- **Data abstraction** is **hiding** the **complex data structure** in order **to simplify** the **user's interface of the system**.
- It is done because many of the users interacting with the database are not that much computer trained to understand the complex data structures of the database system.

Introduction to Database System

View of Data –

- **Three levels** of Data Abstraction –
  - Physical Level
  - Logical Level
  - View Level
- The **main objective** of this three level architecture is to have an **effective separation between the user interface and the physical database.**
- The user never has to concerned regarding the internal storage of the database.
- They want simplified interaction with the database system.

Introduction to Database System

View of Data –

- **Physical Level –**
  - **Lowest level** of abstraction.
  - Describes **how the data are actually stored**.
  - Also describes **how the data can be accessed**.
  - **Describes complex low−level data structures** in detail.
  - **Blocks of consecutive storage locations-** words, byte, etc.
  - Only the **database administrator operates at this level.**

Introduction to Database System

View of Data –

- **Logical Level –**
  - This level is **above the physical level** of abstraction.
  - Describes **what data are stored in the database and what relationship exists among those data.**
  - **Describes entire database** in terms of a **small number of relatively simple structures.**
  - **Do not bother with complex physical storage.**
  - **Database administrator** uses logical level, who decides what information to keep in database.
  - **Programmers/developers** also **work at this level.**

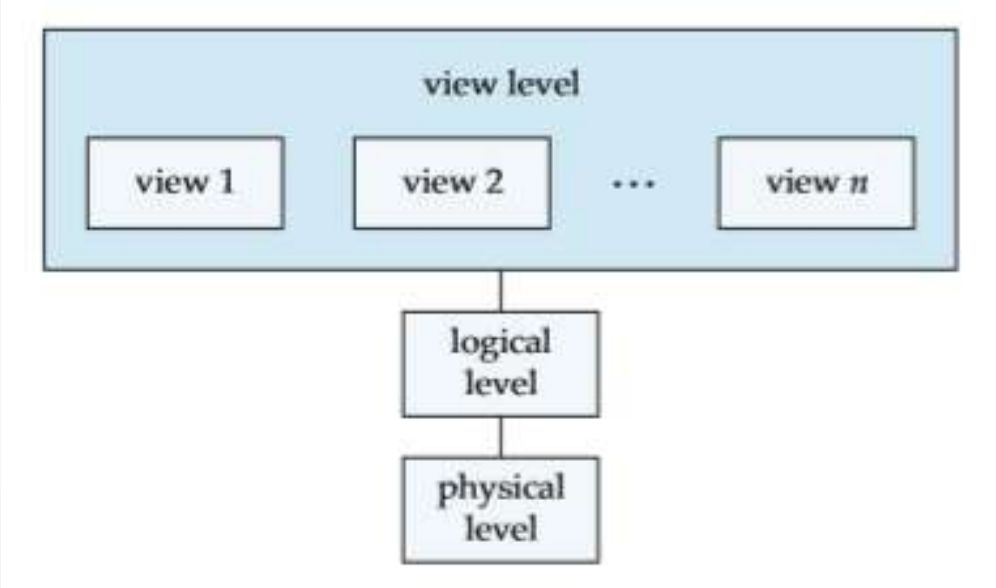
Introduction to Database System

View of Data –

- **View Level –**
  - **Highest level** of abstraction.
  - Describes **only part of entire database**
  - It **shows the data in which** the **user is interested.**
  - **System** may **provide many views for the same data**
  - The **user retrieves the information using different application** from the database.

Introduction to Database System

View of Data –



Introduction to Database System

View of Data –

- **Example –**
  - Suppose we have to create a database of a college. Student, Professor, Department, Course, etc. are the entity sets.
  - The **entity sets** will be **stored** in the storage as the **consecutive blocks** of the memory location. This is the **physical or internal level** and is **hidden from programmers but DBA is aware of it.**
  - At **logical level**, the **programmers define** the **entity sets and relationship among these entity sets** using programming language like SQL. The **programmers and DBA operates** at this level.
  - At the **view level**, the **users** have the **set of applications** which they **use to retrieve the data** they are interested in.

Introduction to Database System

View of Data –

- **Instances and Schema –**
  - **Database changes over time** as information is inserted and deleted.
  - The **collection of information stored** in the database **at a particular moment** is called an **instance** of the database.
  - The **overall design of the database** is called the database **schema**. Schemas are **changed infrequently**. It is the **logical structure of** the **database.**
  - This concept can be understood by analogy to a program written in a programming language. A database **schema** corresponds to the **variable declarations** and **values of the variables** correspond to an **instance** of a database schema.

Introduction to Database System

View of Data –

- **Instances and Schema –**
  - **Database systems have several schemas**, **partitioned according** to the **levels of abstraction**.
  - The **physical schema** describes the **database design at the physical level**.
  - The **logical schema** describes the **database design at the logical level.**
  - A database may also have **several schemas at the view level**, sometimes called **subschemas**, that **describe different views** of the database.

Introduction to Database System

View of Data –

- **Physical Data Independence –**
  - The **logical schema** are **most important**, since **programmers construct applications** by **using** the **logical schema**.
  - The **physical schema** are **hidden** and can usually be **changed easily without affecting application** programs.
  - **Application programs** are said to be **physical data independent** if they **do not depend on the physical schema.**
  - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.

Introduction to Database System

View of Data –

- **Data Models –**
  - The **underlying structure of a database** is the data model.
  - It is **collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.**
  - A data model **provides a way to describe the design of a database** at the physical, logical, and view levels.
  - The data models can be classified into four different categories:
    - Relational Model
    - Entity-Relationship Model
    - Object-based Data Model
    - Semi-structured Data Model

Introduction to Database System

View of Data –

- **Data Models –**
  - **Relational Model -**
    - It uses a **collection of tables** to **represent both data** and the **relationships among those data.**
    - Each **table has multiple columns**, and each **column has a unique name. Tables** are also **known as relations.**
    - It is an example of a **record-based model**. Record-based models are **structured in fixed-format records of several types.**
    - Each **table contains records of a particular type**. Each **record type defines a fixed number of fields**, or attributes.
    - The columns of the table correspond to the attributes of the record type.
    - The **relational data model** is the **most widely used data model**.

Introduction to Database System

View of Data –

- **Data Models –**
  - **Entity-Relationship (E-R) Model -**
    - The E-R data model uses a **collection of basic objects, called entities, and relationships among these objects.**
    - An **entity is a "thing" or "object" in the real world** that is **distinguishable from other objects.**
    - The entity-relationship model is **widely used in database design**.

Introduction to Database System

View of Data –

- **Data Models –**
  - **Object-based Data Model -**
    - Object-oriented programming has become the dominant software-development methodology.
    - This led to the development of an **object-oriented data model** that can be seen as **extending the E-R model with notions of encapsulation, methods (functions), and object identity.**
    - The object-relational data model combines features of the object-oriented data model and relational data model.

Introduction to Database System

View of Data –

- **Data Models –**
  - **Semi-structured Data Model -**
    - It permits the **specification of data** where individual **data items of the same type may have different sets of attributes.**
    - This is in contrast to the data models mentioned earlier, where every data item of a particular type must have the same set of attributes.
    - The **Extensible Markup Language** (XML) is **widely used to represent semi structured data.**

Introduction to Database System

Relational Database –

- **A relational database** is **based on** the **relational model** and **uses a collection of tables to represent** both **data and** the **relationships among those data.**
- It also **includes a DML and DDL.**
- **Tables** –
  - Each table has multiple columns and each column has an unique name.

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Introduction to Database System

Relational Database –

- **Tables –**
  - It is an **example of a record-based model.**
  - Each **table contains records** of a particular type. Each **record type defines a fixed number of fields**, or attributes.
  - The **columns of the table correspond to the attributes** of the record type.
  - It is possible to **create schemas in the relational model** that have problems such as unnecessarily duplicated information.
  - The **relational model hides** such **low-level implementation details from database developers and users.**

Introduction to Database System

Database Architecture –

- The **architecture of a database system** is greatly **influenced by** the **underlying computer system on which the database system runs.**
- Database systems can be **centralized, or client-server**, where one server machine executes work on behalf of multiple client machines.
- Most users of a database system today are not present at the site of the database system, but connect to it through a network.
- This can be differentiated between client machines, on which remote database users work, and server machines, on which the database system runs.
- **Database applications** are usually **partitioned into two or three parts.**

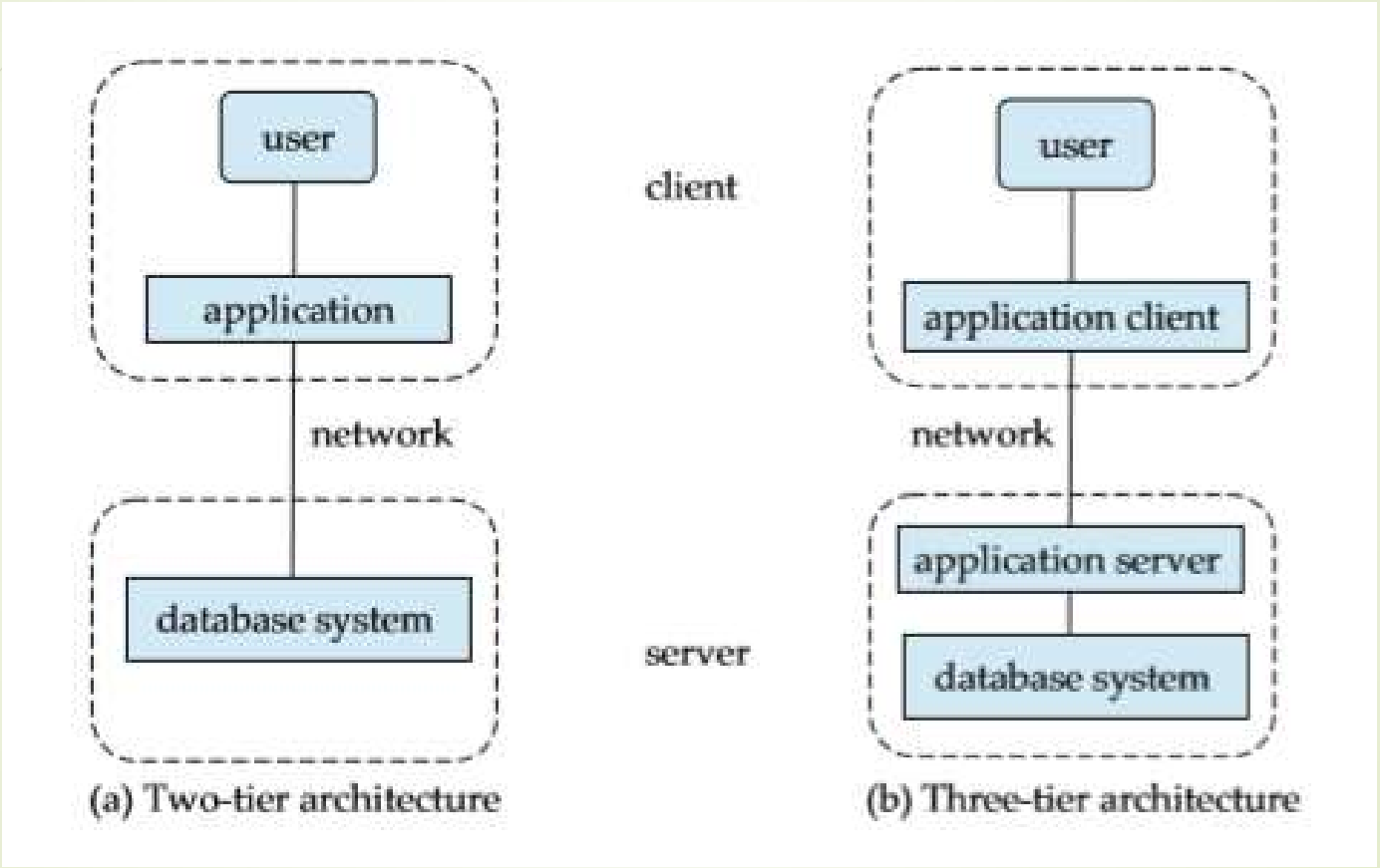
Introduction to Database System

Database Architecture –

- **Two-tier architecture –**
  - In a **two-tier architecture**, the **application resides at the client machine**.
  - It **invokes database system functionality** at the server machine **through query language statements.**
  - Application program interface standards like **ODBC and JDBC are used for interaction between the client and the server.**

Introduction to Database System

Database Architecture –



Introduction to Database System

Database Architecture –

- **Three-tier architecture -**
  - In a **three-tier architecture**, the **client machine acts as a front end** and **does not contain any direct database calls**.
  - Instead, the **client end communicates with an application server**, usually **through a forms interface**.
  - The **application server** in turn **communicates with a database system to access data.**
  - The business logic of the application, which says what actions to carry out under what conditions, is embedded in the application server, instead of being distributed across multiple clients.
  - **Three-tier applications** are more **appropriate for large applications,** and for applications that run on the World Wide Web.

Introduction to Database System

Question Bank -

1) What is Database Management System? Explain the primary goals of DBMS.
2) What is Database Management System? Give its applications.
3) Explain Traditional File Processing System.
4) What is Database Management System? Explain the purpose of DBMS.
5) Compare Database Management System and Traditional File Based System.
6) Explain advantages and disadvantages of DBMS.
7) What is the view of data? Explain different levels of data abstraction.
8) Explain different levels of data abstraction with example.
9) Explain the terms - i) View of data ii) Instances and Schema
10) Explain Data models and its different categories.
11) Explain two-tier and three-tier database architecture.

Introduction to Databases & E-R Model

Mr. S.K.Patil

E-R Model

The Entity-Relationship (E-R) Model –

- The **entity-relationship (E-R) data model** was **developed to facilitate database design.**
- It **allows specification of** an enterprise **schema** that **represents the overall logical structure of a database.**
- The E-R model is very **useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema.**
- The E-R data model **employs three basic concepts**: **entity sets, relationship sets, and attributes.**
- The **E-R model** also **has an associated diagrammatic representation**, the **E-R diagram.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Entity & Entity Sets –**
  - An **entity** is a "**thing**" or **"object"** in the real world that is **distinguishable from all other objects. e.g.– student in college is an entity.**
  - An entity has a **set of properties**, and the **values for some set of properties may uniquely identify an entity**. **e.g.** a student have registration_id property whose value uniquely identifies that person.
  - An **entity** may be **concrete**, such as a **person** or it may be **abstract**, such as a **flight reservation**.
  - An **entity set** is a **set of entities** of the **same type** that **share the same properties, or attributes. e.g.** set of all students in a college
  - Entity sets **do not need to be disjoint**. **e.g.** entity set of all people in university (person). A person entity may be an instructor entity, a student entity, both, or neither.

E-R Model

The Entity-Relationship (E-R) Model –

- **Entity & Entity Sets –**
  - A **database includes a collection of entity sets**, **each** of which **contains any number of entities of the same type**.
- **Relationship Sets –**
  - A **relationship** is an **association among several entities**. **e.g.** relationship **advisor** that **associates instructor with student**.
  - The **association between entity sets** is referred to as **participation**.
  - The entity sets E1, E2, . . .,En participate in relationship set R.
  - A **relationship set** is a **set of relationships of the same type**.
  - The **function that** an **entity plays in a relationship** is called that **entity's role.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Attributes –**
  - An **entity is represented** by a set of **attributes.**
  - **Attributes** are **descriptive properties possessed by each member** of an entity set.
  - **Each entity has a value for each of its attributes**. e.g. a student entity may have the value 123 for Roll no, the value ABC for Name, the value M for Gender, and the value 18 for Age.
  - **Some attributes** are used to **uniquely identify entities**. e.g. **Roll no** in previous
  - **For each attribute**, there is a **set of permitted values**, called the **domain, or value set**, of that attribute.
  - A **database** thus **includes** a **collection of entity sets**, each of which **contains any number of entities** of the **same type.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Attributes –**
  - An **attribute** of an entity set is a **function that maps** from the **entity set into** a **domain.**
  - Since an **entity set may have several attributes**, **each entity** can be **described by a set of (attribute, data value) pairs**, **one pair for each attribute of the entity set.**
  - e.g. (customer-id, 111111) , (customer-name, ABC)
  - An **attribute** can be **characterized by the following attribute types** –
    - Simple and composite attributes
    - Single-valued and multivalued attributes
    - Derived attribute

E-R Model

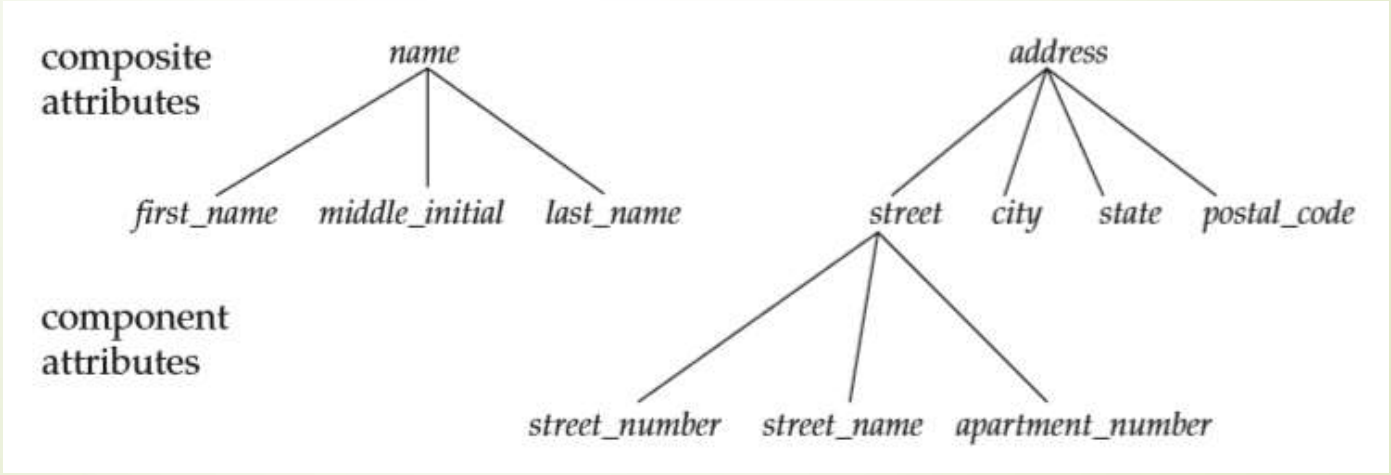
The Entity-Relationship (E-R) Model –

- **Attributes –**
  - **Simple and composite attributes –**
    - **Simple attributes** are **not divided into subparts**.
    - **Composite attributes** can be **divided into subparts** (other attributes).
    - **e.g.** an attribute **name** could be **structured as a composite attribute** consisting of **first-name, middle_initial, and last-name**. (address?)
    - **Using composite attributes in a design schema** is a **good choice** if a **user** will **wish to refer to an entire attribute on some occasions**, and to **only a component of the attribute on other occasions**.
    - **Composite attributes help** us to **group together related attributes**, **making** the **modeling cleaner**.
    - **Composite attribute** may **appear as a hierarchy**.

E-R Model

The Entity-Relationship (E-R) Model –

- **Attributes –**
  - **Simple and composite attributes –**
    - In the **composite attribute address**, its **component attribute street** can be further divided into street number, street name, and apartment number.



E-R Model

The Entity-Relationship (E-R) Model –

- **Attributes –**
  - **Single-valued and multivalued attributes –**
    - The **attributes which have a single value for a particular entity** are said to be **single valued.**
    - e.g. the **student_ID** attribute for a specific student entity refers to only one student ID.
    - The **attributes which have a set of values for a specific entity** are said to be **multivalued.**
    - e.g. employee entity set with the attribute **phone-number**. An **employee may have zero, one, or several phone numbers** and different employees may have different numbers of phones.

E-R Model

The Entity-Relationship (E-R) Model –

- **Attributes –**
  - **Derived attribute –**
    - The **value for this type of attribute** can be **derived from the values of other related attributes or entities.**
    - **e.g.** the **customer entity** set **has an attribute loans-held**, which **represents how many loans a customer has from the bank**. We can **derive the value** for this attribute **by counting the number of loan entities associated with that customer.**
    - The **value of a derived attribute is not stored**, but is **computed when required.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Attributes –**
  - An **attribute takes** a **null value when** an **entity does not have a value** for it.
  - The **null value** may **indicate "not applicable" -** the value does not exist for the entity.
  - **e.g. one may have no middle name**
  - **Null** can **also designate** that an **attribute value is unknown**.
  - An **unknown value** may be **either missing** (the **value does exist, but we do not have** that **information**) **or not known** (we **do not know whether or not the value actually exists**).

E-R Model

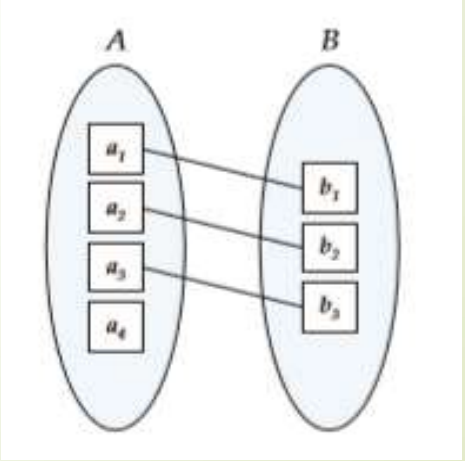
The Entity-Relationship (E-R) Model –

- **Constraints –**
  - An **E-R enterprise schema** may **define certain constraints to which** the **contents of a database must conform**.
  - The **most important types of constraints** : **mapping cardinality & participation constraint**
  - **Mapping Cardinalities –**
    - **Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set.**
    - **e.g. each account** must **belong to only one customer**
    - Mapping cardinalities are **most useful in describing binary relationship sets** (relationship between two entity sets).

E-R Model

The Entity-Relationship (E-R) Model –

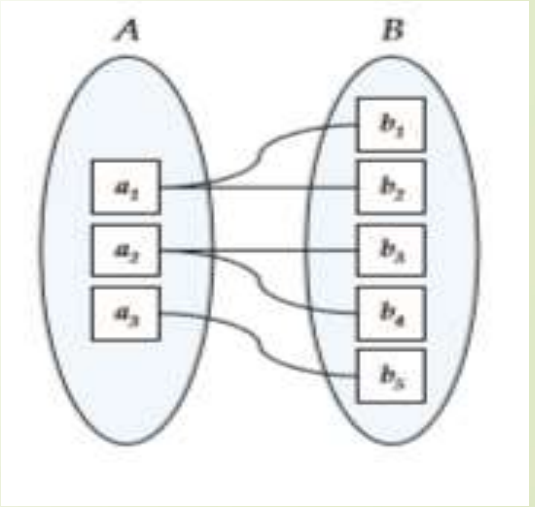
- **Constraints –**
  - **Mapping Cardinalities –**
    - For a binary relationship set R between entity sets A and B, the **mapping cardinality must be one of the following:**
    - **One-to-one :** An **entity in A** is **associated with at most one entity in B**, and an **entity in B** is **associated with at most one entity in A**.
    - **e.g. One Customer – one account**



E-R Model

The Entity-Relationship (E-R) Model –

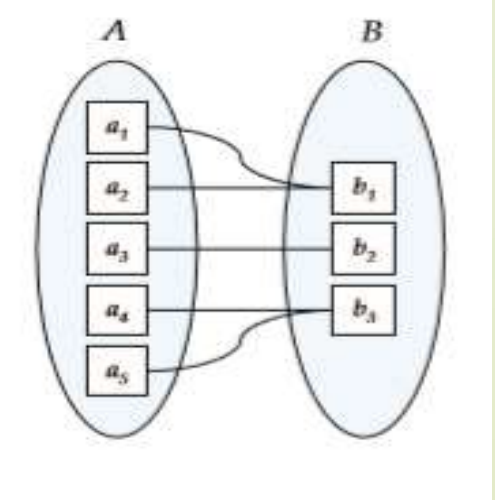
- **Constraints –**
  - **Mapping Cardinalities –**
    - **One-to-many :** An **entity in A** is **associated with any number (zero or more) of entities in B**. An **entity in B**, however, can be **associated with at most one entity in A.**
    - **e.g. One Customer – many loan accounts**



E-R Model

The Entity-Relationship (E-R) Model –

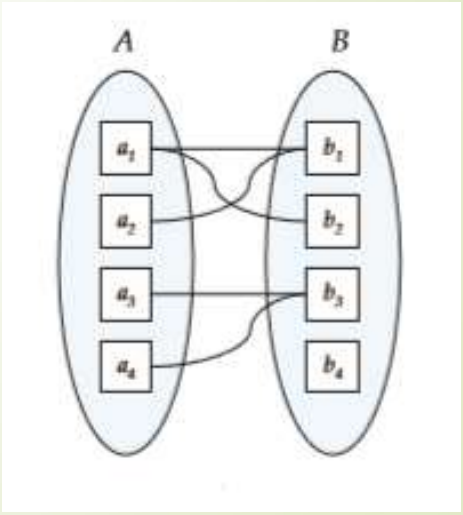
- **Constraints –**
  - **Mapping Cardinalities –**
    - **Many-to-one :** An **entity in A** is **associated with at most one entity in B**. An **entity in B**, however, can be **associated with any number (zero or more) of entities in A.**
    - **e.g. 3 Customers - joint loan account**



E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Mapping Cardinalities –**
    - **Many-to-many :** An **entity in A** is **associated with any number (zero or more) of entities in B**, and an **entity in B** is **associated with any number (zero or more) of entities in A.**
    - **e.g. Many customers many loan accounts**



E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Participation Constraints –**
    - The **participation of an entity set E in a relationship set R** is **said to be total** if **every entity in E participates** in **at least one relationship in R.**
    - If only **some entities in E participate in relationships in R**, the **participation of entity** set E in relationship R is **said to be partial.**
    - In Figure one-to-one, the participation of B in the relationship set is total while the participation of A in the relationship set is partial.
    - In Figure one-to-many, the participation of both A and B in the relationship set are total.

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - An **attribute or set of attributes** whose **values uniquely identify each entity** in an entity set is called as **key** for that entity set.
    - Any key consisting of single **attribute** is called a **simple key** while that consisting of a **combination of attributes** is called a **composite key.**
    - **No two entities** in an entity set are **allowed to have exactly the same value** for all attributes.
    - A **key allows** us to **identify a set of attributes** that suffice to **distinguish entities from each other.**
    - The concepts of **superkey, candidate key, and primary key** are **applicable to entity sets.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - **Superkey –**
      - A **superkey** is a **set of one or more attributes** that, taken collectively, **allow us to identify uniquely an entity** in the entity set.
      - **e.g.** the **customer-id** attribute of the entity set customer is **sufficient to distinguish one customer entity from another**. Thus, customer-id is a superkey.
      - **Similarly**, the **combination of customer-name and customer-id** is a **superkey** for the entity set customer. The **customer-name** attribute of customer **is not a superkey**, because **several people might have the same name.**
      - It supports NULL values.

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - **Candidate key –**
      - A **candidate key** is a **subset of super key**. It is the **minimal set of attributes** that can **uniquely identify an entity**.
      - **Several distinct sets of attributes** could **serve as a candidate key**.
      - It is a **single field** or the **list combination of fields** that **uniquely identifies each record.**
      - Suppose a **combination of customer-name and customer-street** is sufficient to distinguish among members of the customer entity set. Then, both {customer-id} and {customer-name, customer-street} are candidate keys.

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - **Candidate key –**
      - Although the **attributes customer-id and customer-name together can distinguish customer entities, but their combination does not form a candidate key.** Since the **attribute customer-id alone is a candidate key.**
      - The value of the Candidate Key is unique and may be null for an entity.

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - **Primary key –**
      - Primary key is a **candidate key chosen by the database designer** to **identify entities within an entity set.**
      - A **key** (primary, candidate, and super) is a **property of the entity set**, rather than of the individual entities.
      - **Any two individual entities** in the set are **prohibited** from **having** the **same value on the key attributes at the same time.**
      - **Candidate keys** must be **chosen with care**. The **primary key** should be **chosen such that** its **attributes are never, or very rarely, changed.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - **Primary key –**
      - **e.g.** the **address field** of a student **should not be** part of the **primary key**, since it is **likely to change. Roll no**, on the other hand, are **guaranteed to never change**.
      - The **primary key** of a **relational table uniquely identifies each record** in the table.
      - It can **either be a normal attribute** that is guaranteed to be unique (such as Roll no) or it can be **generated by the DBMS** (such as a globally unique identifier, or GUID, in Microsoft SQL Server).
      - **Primary keys may consist** of a **single attribute or multiple attributes in combination.**

E-R Model

The Entity-Relationship (E-R) Model –

- **Constraints –**
  - **Keys –**
    - **Foreign key –**
      - The **primary key of one entity set used as an attribute in another entity set** then that key is called Foreign key.
      - e.g. We add the primary key of the Student entity , student_id, as a new attribute in the Course_enrolled entity.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

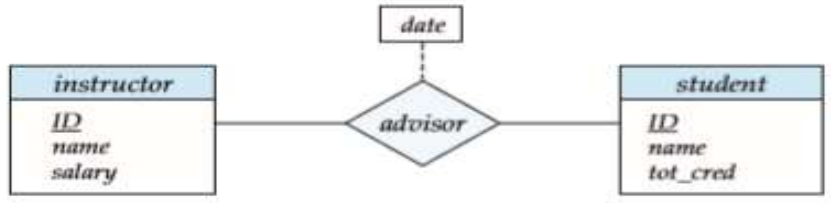
- An **E-R diagram** can **express** the **overall logical structure of a database graphically.**
- E-R diagrams are **simple and clear**.
- **Components of E-R diagram –**
  - **Rectangle –**
    - It **represent** the **entity sets**.
    - It has **two parts** – **first part contains** the **name** of the entity set and **second part contains** the **names of all the attributes** of the entity set.

E-R diagram corresponding to instructors and students.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- **Components of E-R diagram –**
  - **Diamond –** It **represent** the **relationship sets**.
  - **Undivided Rectangles –** It **represent** the **attributes of a relationship set. Attributes that are part of** the **primary key** are **underlined**.
  - **Lines –** It **links entity sets to relationship sets**.
  - **Dashed Lines -** It **links attributes of** a **relationship set to the relationship set**.
  - **Double Line –** It **indicates total participation** of an entity in a relationship set.
  - **Double Diamond –** It **represent identifying relationship** sets **linked to weak entity sets.**



E-R diagram with an attribute attached to a relationship set.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

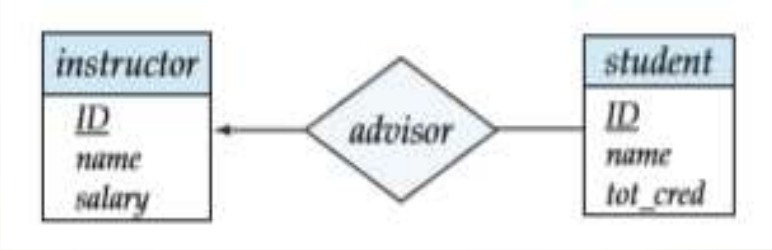
- **Mapping Cardinality –**
  - The **relationship** set between the entity sets **may be one-to-one, one-to-many, many-to-one, or many-to-many.**
  - To **distinguish among these types**, we **draw either** a **directed line(→)** or an **undirected line(—) between** the **relationship set and the entity set**, as follows:
  - **One-to-one** - We draw a **directed line from** the **relationship set to both entity sets.**

  - This **indicates** that an **instructor may advise at most one student**, and a **student may have at most one advisor**.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- **Mapping Cardinality –**
  - **One-to-many** - We draw a **directed line from the relationship set advisor to** the entity set **instructor** and an **undirected line to** the entity set **student**. This **indicates** that an **instructor may advise many students**, but a **student may have at most one advisor.**



  - **Many-to-one** - We draw an **undirected line from the relationship set advisor to the entity set instructor** and a **directed line to** the entity set **student**. This **indicates** that an **instructor may advise at most one student**, but a **student may have many advisors.**

Database Design & E-R Model

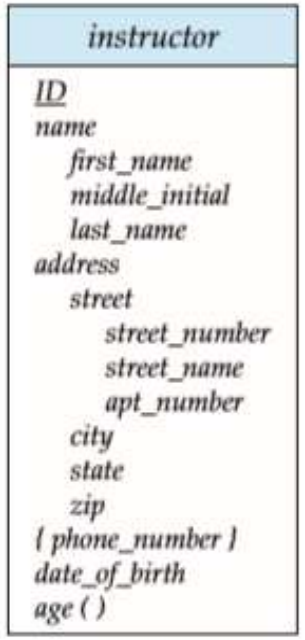
Entity-Relationship Diagram / E-R Diagram –

- **Mapping Cardinality –**
  - **Many-to-many** – We draw an **undirected line from the relationship set advisor to both entity sets** instructor and student. This **indicates** that an **instructor may advise many students, and a student may have many advisors.**

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- **Complex Attributes –**
  - Figure shows how composite attributes can be represented in the E-R notation.
  - Here, a composite attribute name, with component attributes first_name, middle_initial, and last_name replaces the simple attribute name of instructor.
  - The address can be defined as the **composite attribute** with the attributes street, city, state, and zip code.
  - Figure also illustrates a **multivalued attribute** phone_number denoted by **"{phone number}"** and a **derived attribute** age, shown by a "**age ()**".



E-R Model

Entity-Relationship Diagram / E-R Diagram –

- **Weak Entity Sets –**
  - An **entity set** that **does not have sufficient attributes to form a primary key** is termed as **weak entity set**.
  - For a weak entity set to be meaningful, it **must be associated with another entity set**, called the **identifying or owner entity set**.
  - **Every weak entity** must be **associated with an identifying entity**; that is, the **weak entity set** is said to be **existence dependent on** the **identifying entity set.**
  - The **identifying entity set** is said to **own the weak entity set that it identifies.**
  - The **relationship associating** the **weak entity set with** the **identifying entity set** is called the **identifying relationship.**

E-R Model

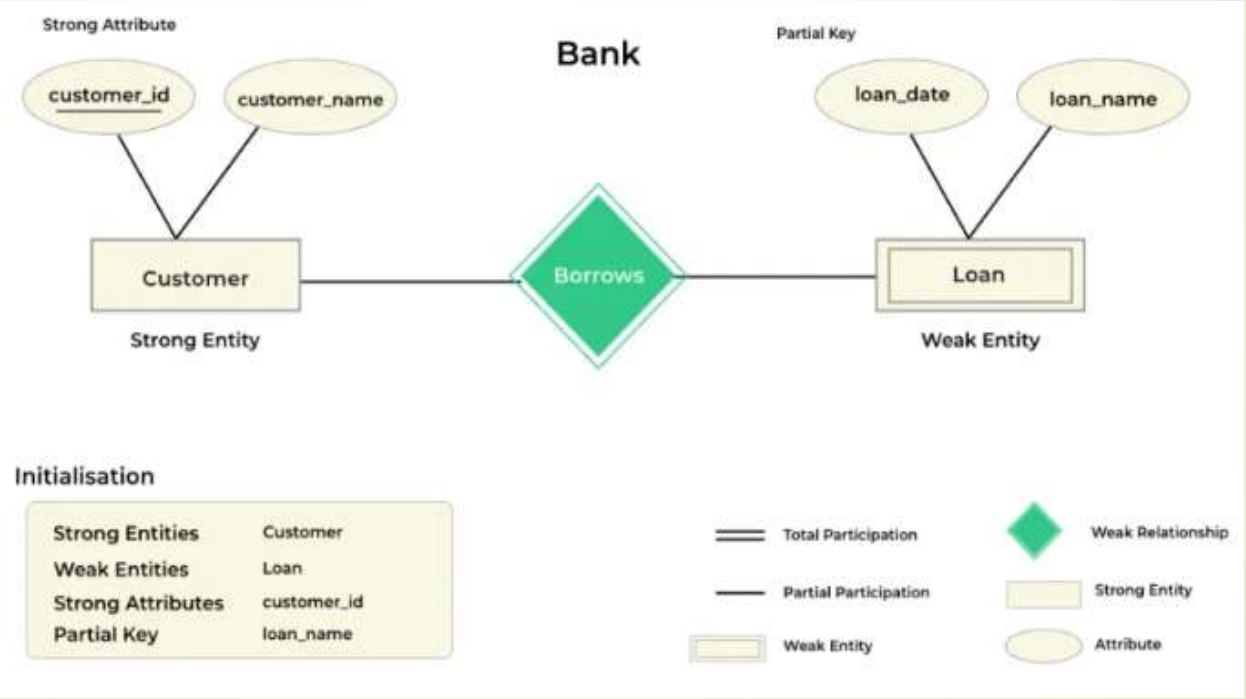
Entity-Relationship Diagram / E-R Diagram –

- **Weak Entity Sets –**
  - The **identifying relationship** is **many-to-one from** the **weak entity set to** the **identifying entity set**, and the **participation of the weak entity set** in the relationship is **total**.
  - The **weak entity set contains** a **partial key** called a **discriminator** which **helps in identifying a group of entities from the entity set.**
  - In E-R diagram –
    - A **discriminator represented by a dashed line**
    - A **weak entity set represented by double rectangle**
    - A **relationship between weak and strong entity set** (identifying relationship) **represented by double diamond**.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- **Weak Entity Sets and Strong Entity Sets –**
  - The **primary key of a weak entity set** is **formed by** the **primary key of** the **identifying entity set**, **plus** the **weak entity set's discriminator**.



E-R Model

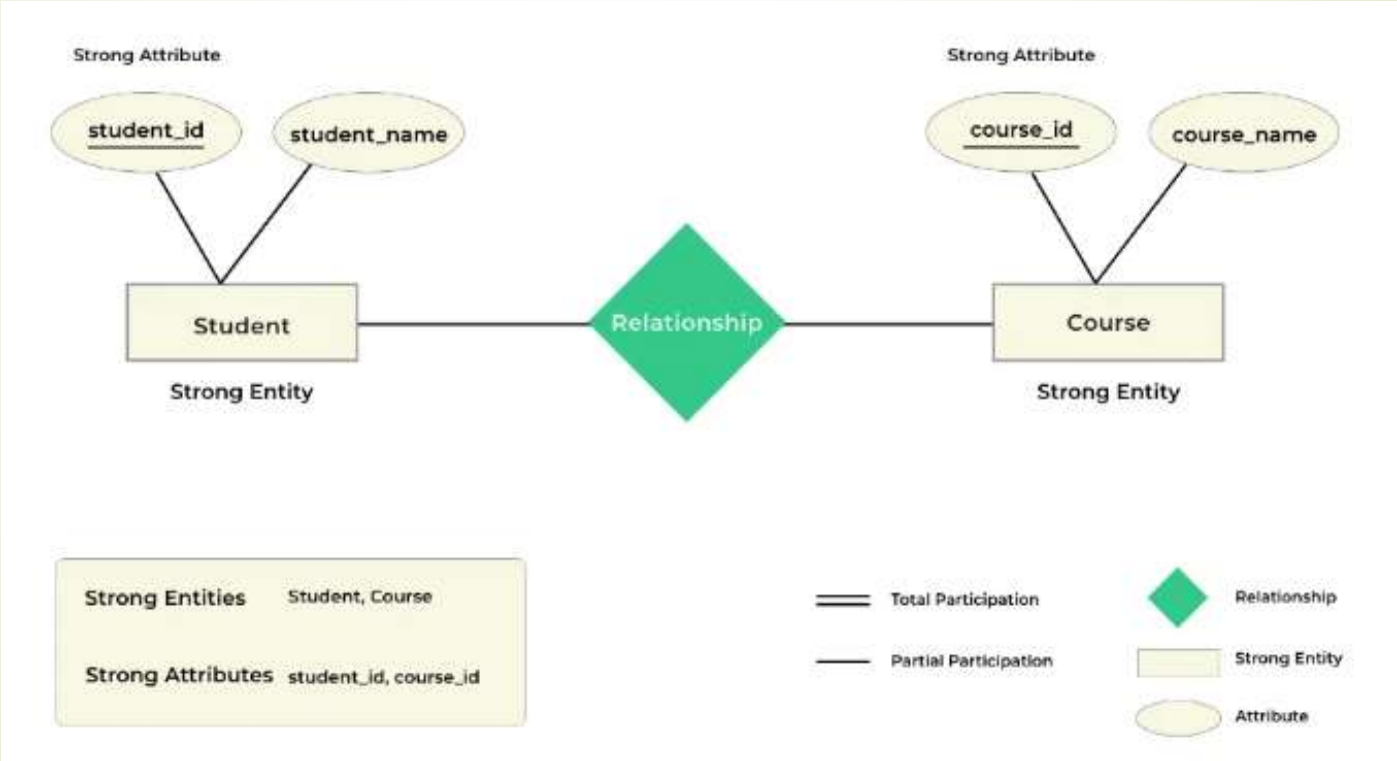
Entity-Relationship Diagram / E-R Diagram –

- **Strong Entity Sets –**
  - An **entity set that has a primary key** is termed as **strong entity set**.
  - Strong Entity is **independent of any other entity** in the schema.
  - The **strong entity** is represented by a **single rectangle.**
  - The **relationship between two strong entities** is represented by a **single diamond.**
  - The **primary key of the strong entity** is represented by **underlining it** (Called as **Strong Attribute**).

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- **Strong Entity Sets –**



E-R Model

Entity-Relationship Diagram / E-R Diagram –

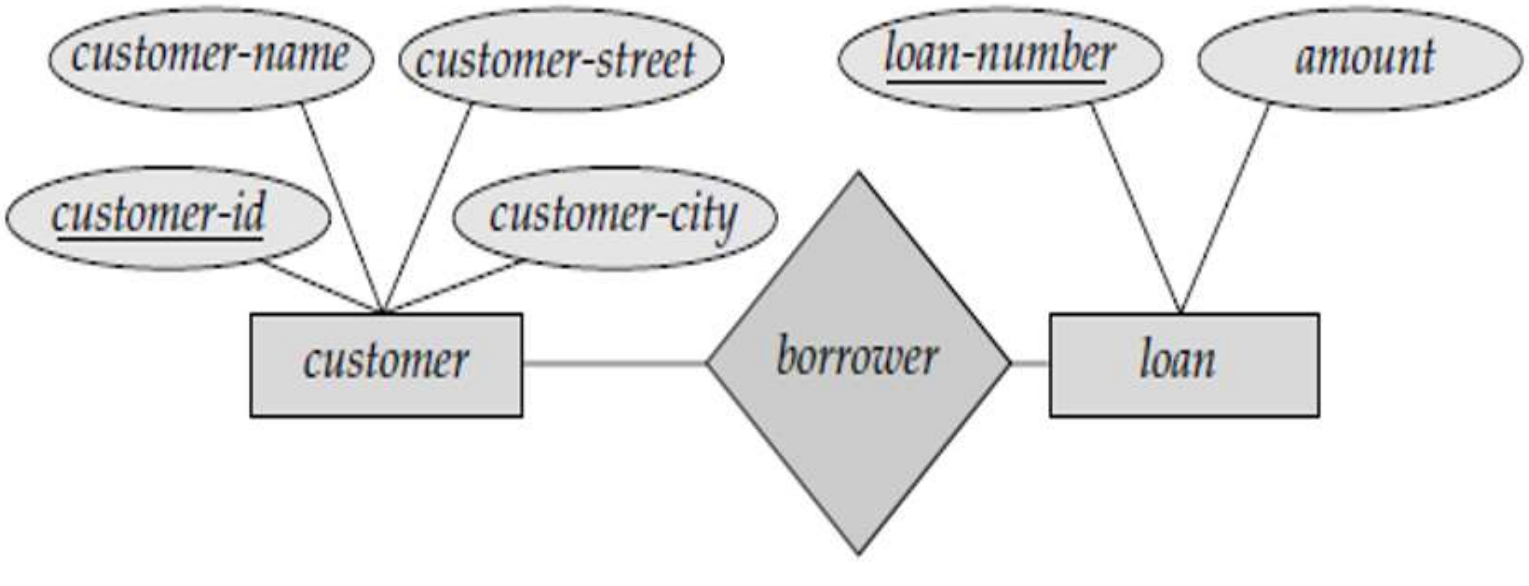
- **Strong Entity Set Vs. Weak Entity Set –**

Strong Entity Set	Weak Entity Set
It has a **primary key**.	It will **not** have a **primary key** but it has **partial discriminator key**.
Not dependent on any other entity.	**Dependent** on the strong entity.
Represented by a **single rectangle**.	Represented by **double rectangle.**
Relationship between two strong entities is represented by a **single diamond**.	**Relationship** between a strong entity and the weak entity is represented by **double diamond**.
A strong entity **may or may not** have **total participation**.	It has **always total participation**.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- Sample E-R Diagrams -

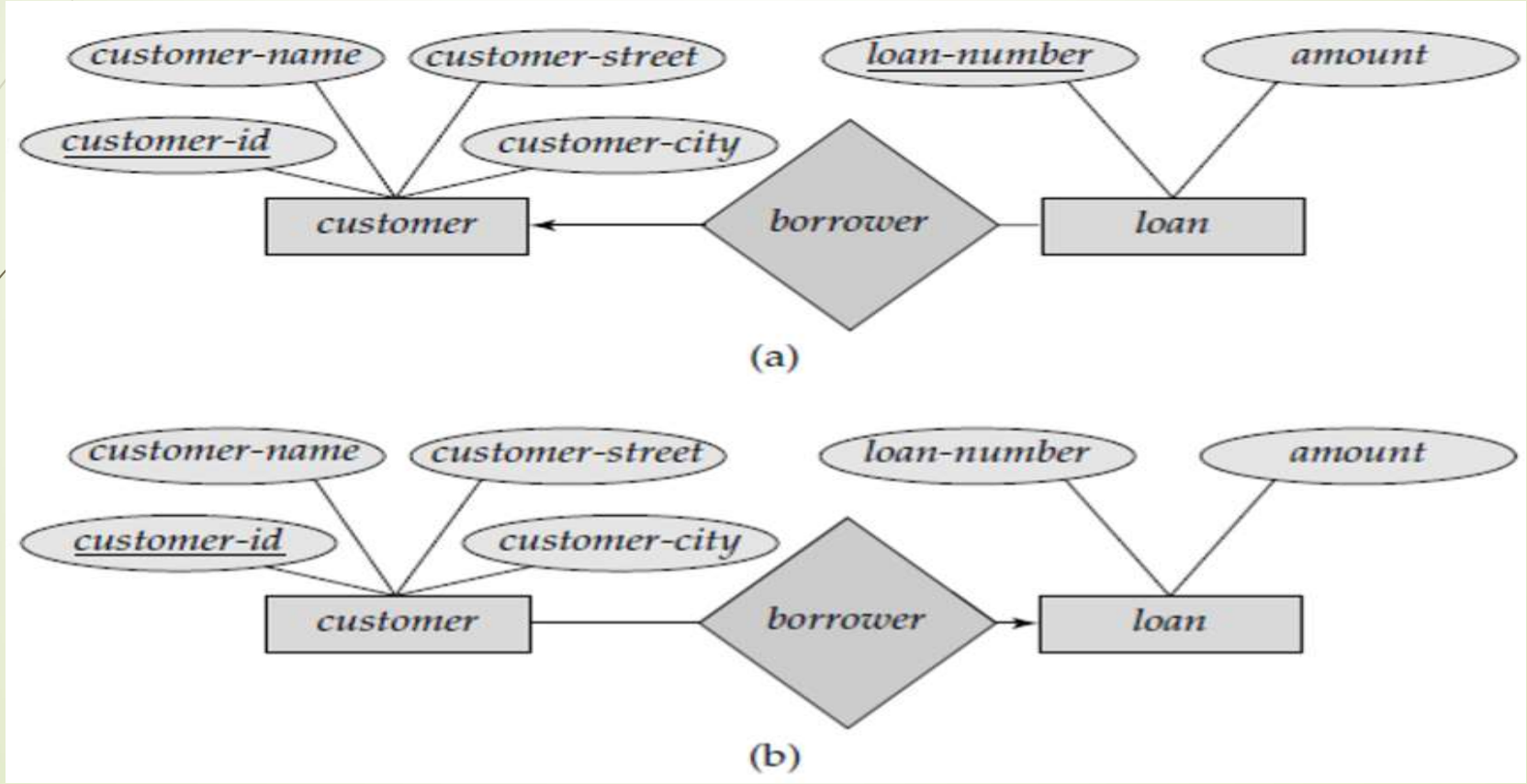


E-R diagram corresponding to customers and loans.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

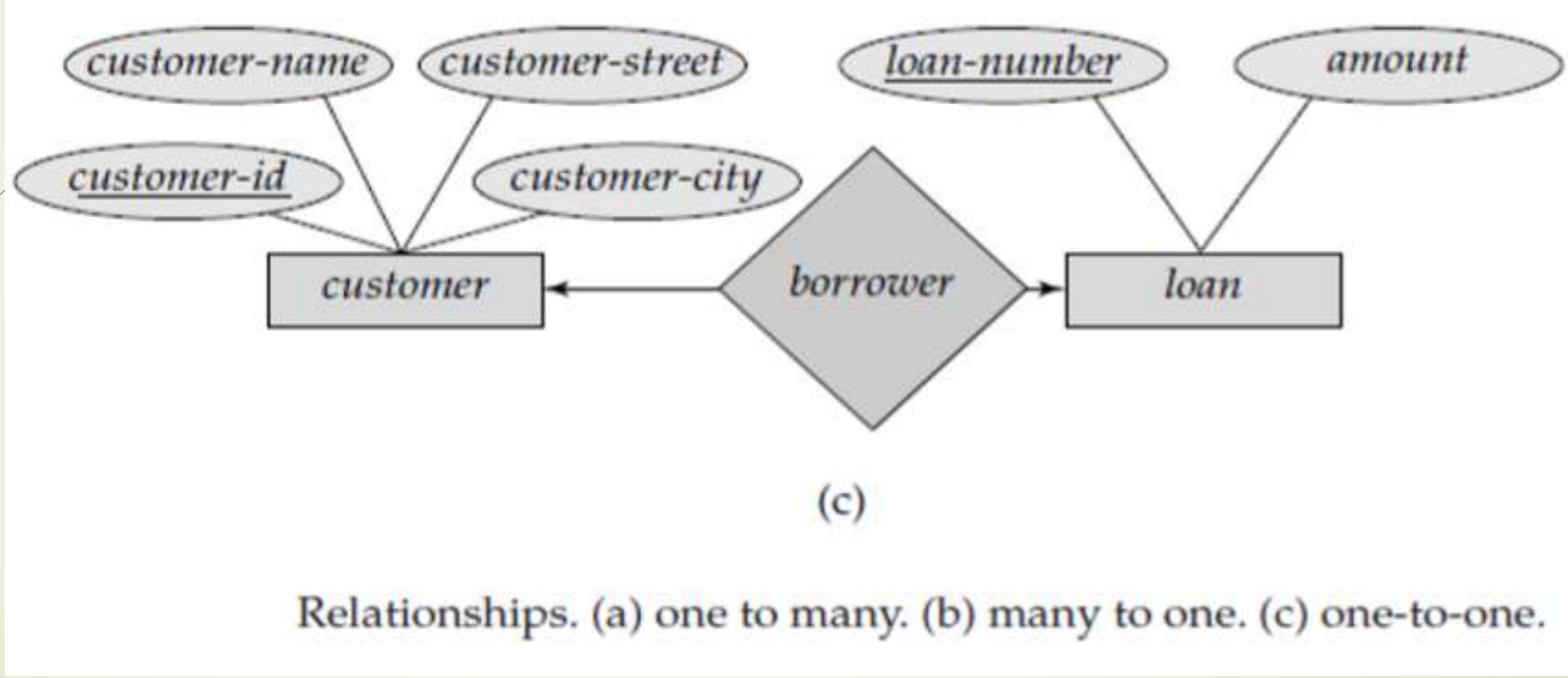
- Sample E-R Diagrams -



E-R Model

Entity-Relationship Diagram / E-R Diagram –

- Sample E-R Diagrams -

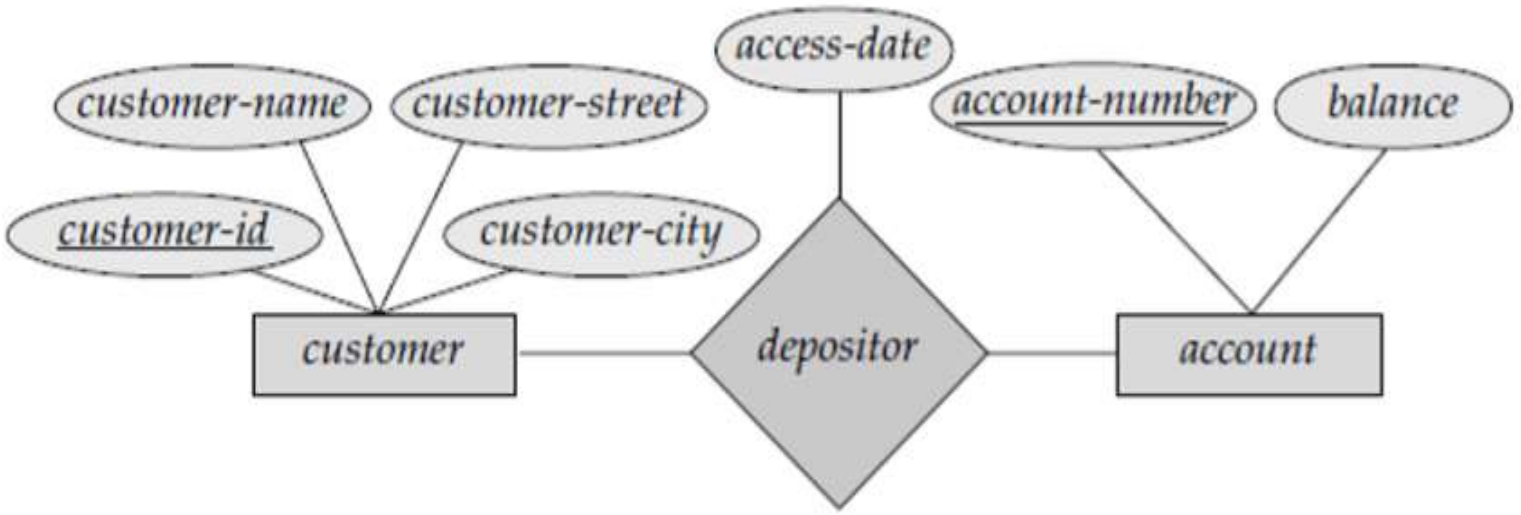


Relationships. (a) one to many. (b) many to one. (c) one-to-one.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- Sample E-R Diagrams -

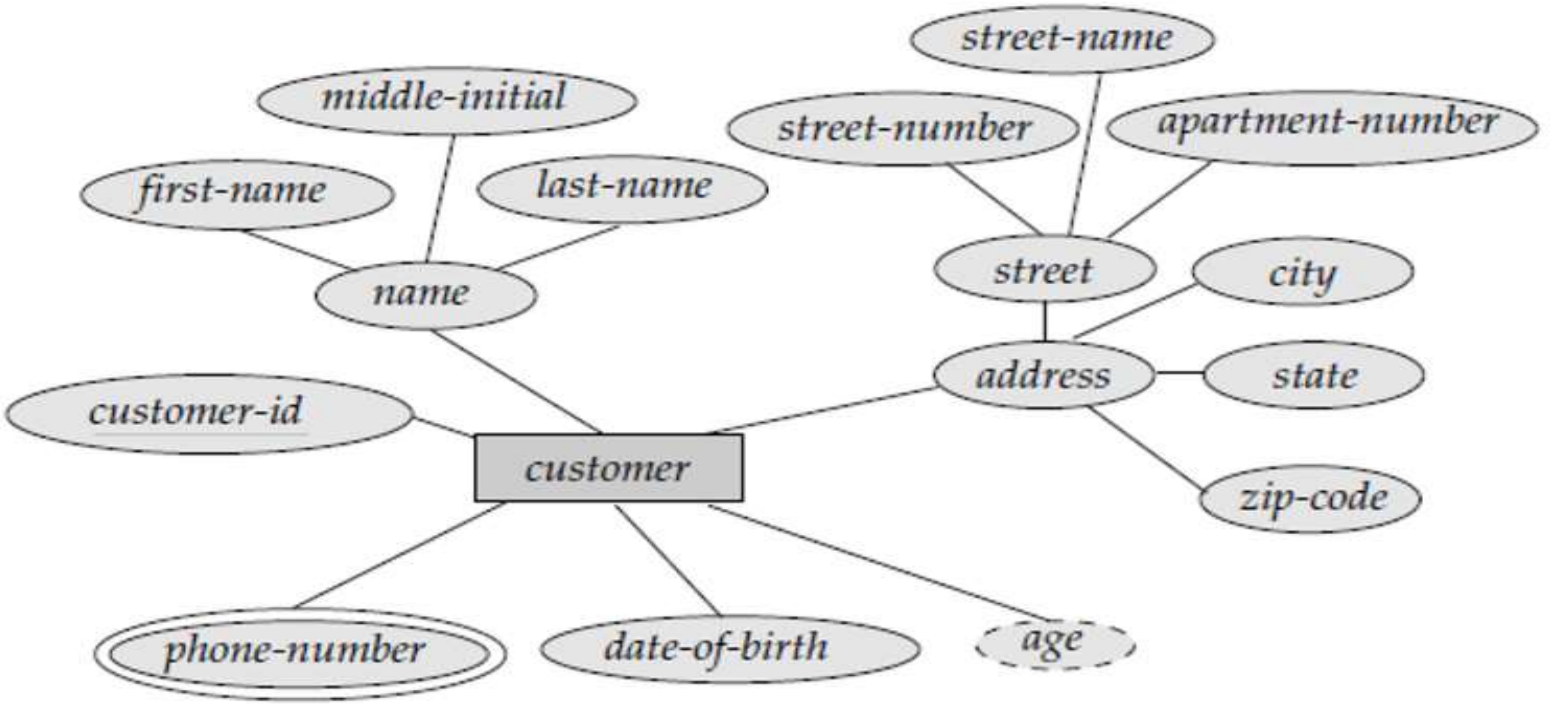


E-R diagram with an attribute attached to a relationship set.

E-R Model

Entity-Relationship Diagram / E-R Diagram –

- Sample E-R Diagrams -

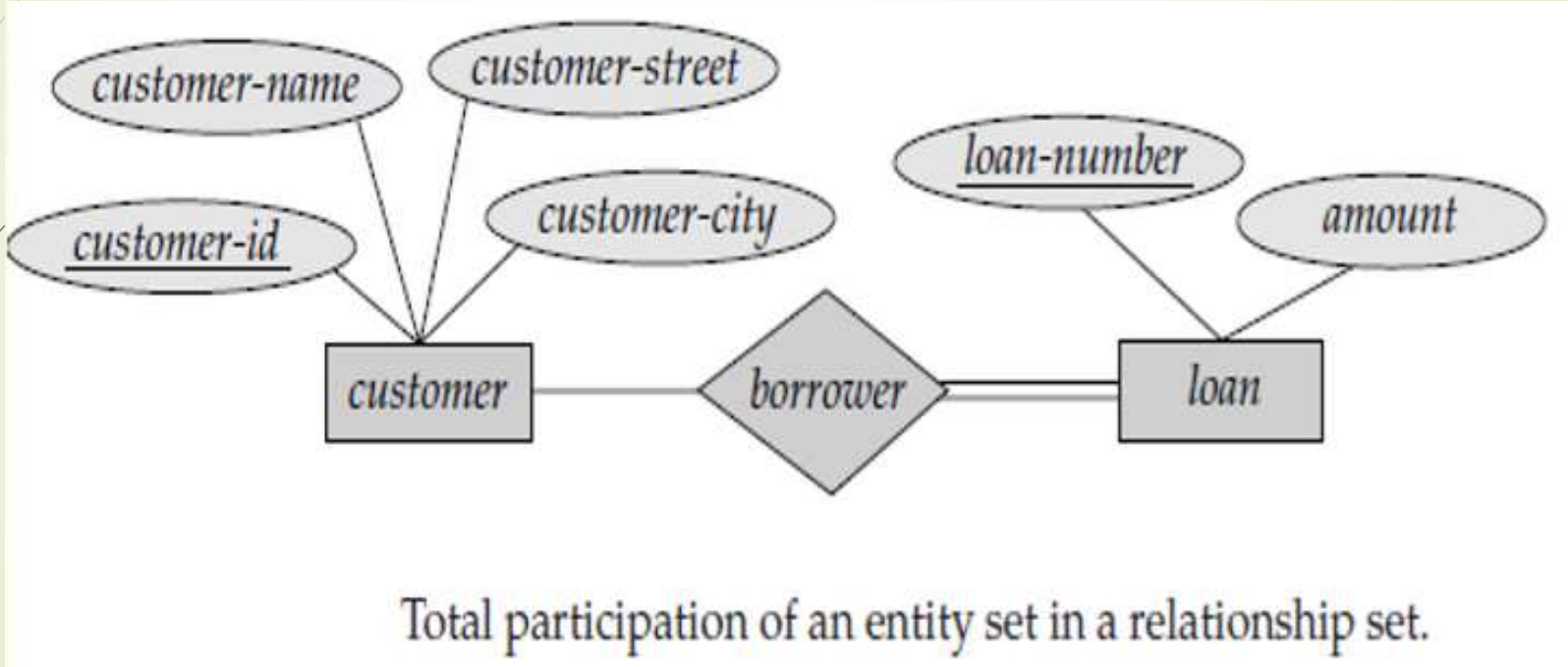


E-R diagram with composite, multivalued, and derived attributes.

Database Design & E-R Model

Entity-Relationship Diagram / E-R Diagram –

- Sample E-R Diagrams -



Total participation of an entity set in a relationship set.

E-R Model

Reduction to Relational Schemas –

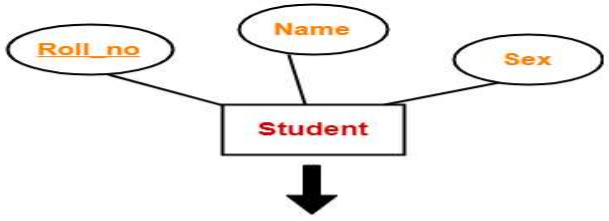
- We can **represent** a database that conforms to an **E-R database schema by a collection of relation schemas.**
- **For each entity set and for each relationship set** in the database design, there is a **unique relation schema** to which we assign the name of the corresponding entity set or relationship set.
- Both the **E-R model** and the **relational database model** are **abstract, logical representations** of real-world enterprises.
- **Because** the **two models employ similar design principles**, we can **convert an E-R design into a relational design.**
- After designing an ER diagram –
  - **ER diagram is converted into the tables in relational model**.
  - This is because relational models can be easily implemented by RDBMS like MySQL, Oracle, etc.

E-R Model

Reduction to Relational Schemas –

Following rules are used for converting an ER diagram into the tables –

- **Rule 1 : For Strong Entity Set with only Simple Attributes –**
  - A **strong entity set with only simple attributes** will **require only one table** in relational model.
    - **Columns of the table** will be the **attributes of the entity set**.
    - The **primary key** of the table will be the **key attribute of the entity set**.



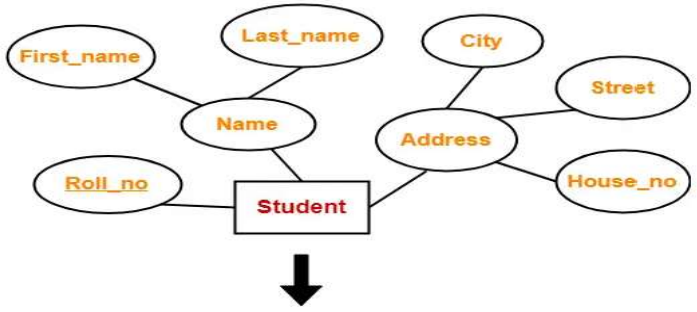
Roll_no	Name	Sex

Schema : Student (Roll_no , Name , Sex)

E-R Model

Reduction to Relational Schemas –

- **Rule 2 : For Strong Entity Set with Composite Attributes –**
  - A **strong entity set with any number of composite attributes** will **require only one table** in relational model.
  - While conversion, **simple attributes** of the composite attributes are **taken into account and not the composite attribute itself.**



Roll_no	First_name	Last_name	House_no	Street	City

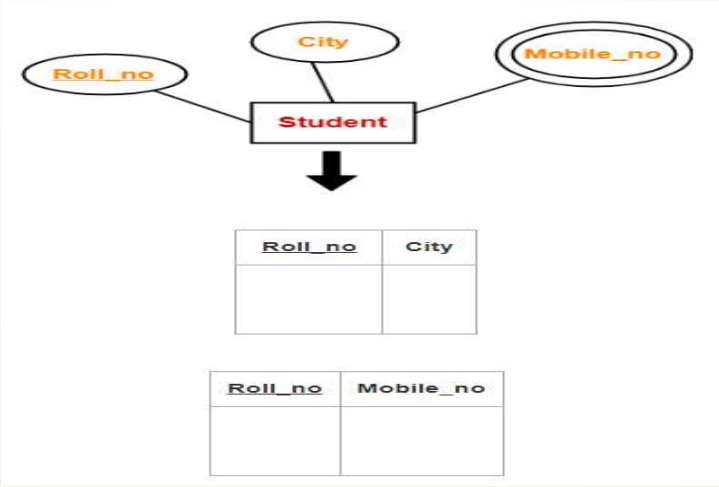
Schema :

Student (Roll_no , First_name , Last_name , House_no , Street , City)

E-R Model

Reduction to Relational Schemas –

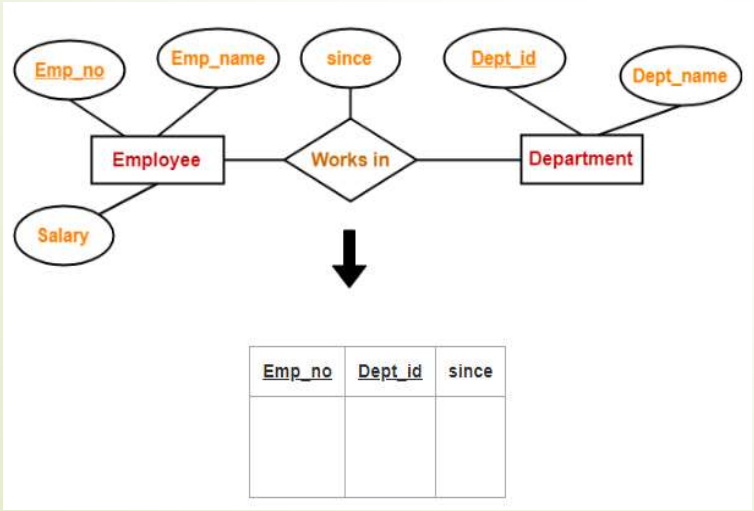
- **Rule 3 : For Strong Entity Set with Multivalued Attributes –**
  - A **strong entity set with any number of multivalued attributes** will **require two tables** in relational model.
    - **One table** will contain all the **simple attributes with the primary key**
    - **Other table** will contain the **primary key and all the multivalued attributes.**



E-R Model

Reduction to Relational Schemas –

- **Rule 4 : Translating Relationship Set into a Table –**
  - A **relationship set** will **require one table** in the relational model.
  - **Columns of the table are** –
    - **Primary key attributes of the participating entity sets**
    - Its **own descriptive attributes** if any.

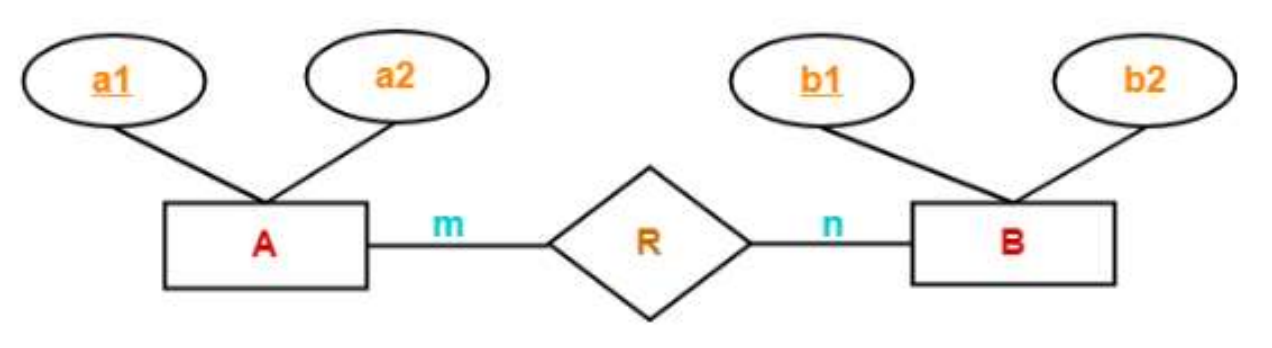


Schema : Works in (Emp_no , Dept_id , since)

E-R Model

Reduction to Relational Schemas –

- **Rule 5 : For Binary Relationships with Cardinality Ratios –**
  - The following four cases are possible –
    - **Case 1 – With Cardinality ratio m:n**

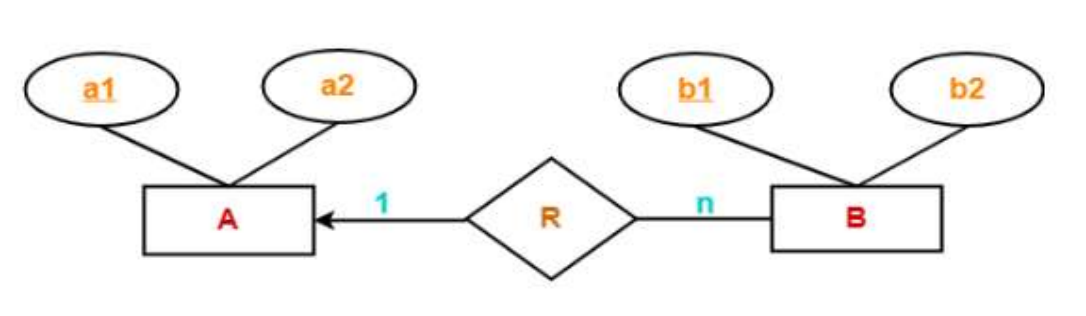


- Here, three tables will be required-
  - A (a1 , a2)
  - R (a1 , b1)
  - B (b1 , b2)

E-R Model

Reduction to Relational Schemas –

- **Rule 5 : For Binary Relationships with Cardinality Ratios –**
  - The following four cases are possible –
    - **Case 2 – With Cardinality ratio 1:n**

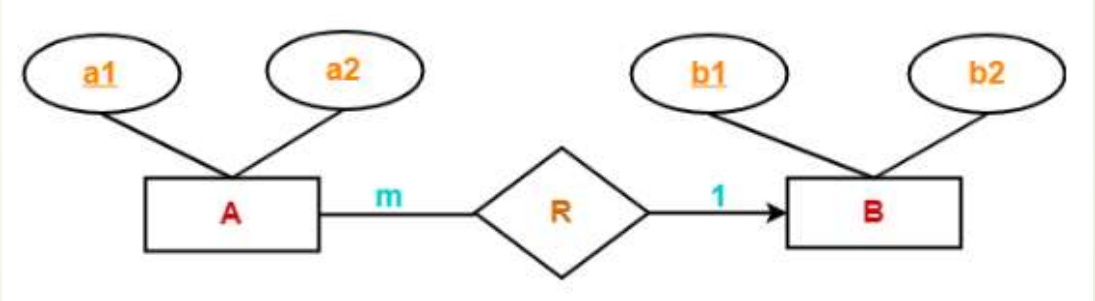


- Here, two tables will be required-
  - A (a1 , a2)
  - BR (a1 , b1 , b2)

E-R Model

Reduction to Relational Schemas –

- **Rule 5 : For Binary Relationships with Cardinality Ratios –**
  - The following four cases are possible –
    - **Case 3 – With Cardinality ratio m:1**

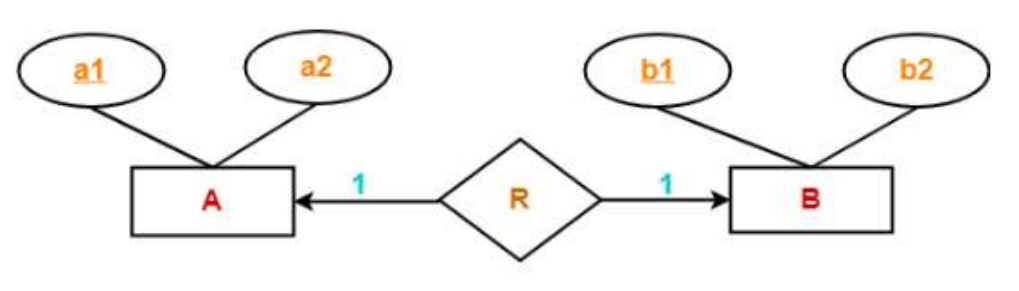


- Here, two tables will be required-
  - AR (a1 , a2 , b1)
  - B (b1 , b2)

E-R Model

Reduction to Relational Schemas –

- **Rule 5 : For Binary Relationships with Cardinality Ratios –**
  - The following four cases are possible –
    - **Case 4 - With Cardinality ratio 1:1**



- Here, two tables will be required. Either combine 'R' with 'A' or 'B'
  - Way-01: AR (a1 , a2 , b1) , B (b1 , b2)
  - Way-02: A (a1 , a2), BR (a1 , b1 , b2)

E-R Model

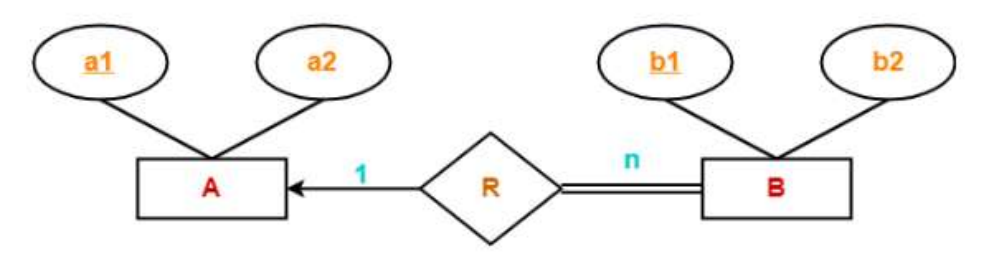
Reduction to Relational Schemas –

- **Rule 5 : For Binary Relationships with Cardinality Ratios –**
  - **Thumb Rules to Remember** – While **determining the minimum number of tables required for binary relationships** with given cardinality ratios, following thumb rules must be kept in mind-
    - For binary relationship with **cardinality ration m : n** , **separate and individual tables** will be drawn **for each entity set and relationship**.
    - For binary relationship with **cardinality ratio either m : 1 or 1 : n** , always remember "**many side will consume the relationship**" i.e. a **combined table** will be drawn for **many side entity set and relationship set**.
    - For binary relationship with **cardinality ratio 1 : 1** , **two tables** will be required. You can **combine the relationship set with any one** of the entity sets.

E-R Model

Reduction to Relational Schemas –

- **Rule 6 : For Binary Relationships with Both Cardinality Constraints & Participation Constraints –**
  - **Case 1 - With Cardinality constraint and total participation from one side –**

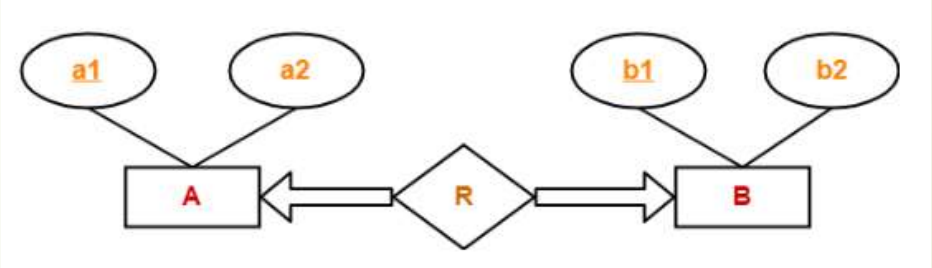


  - Because cardinality ratio = 1 : n , so we will combine the entity set B and relationship set R. Then, two tables will be required-
    - A (a1 , a2)
    - BR (a1 , b1 , b2)
  - Because of total participation, foreign key a1 has acquired NOT NULL constraint, so it can't be null now.

E-R Model

Reduction to Relational Schemas –

- **Rule 6 : For Binary Relationships with Both Cardinality Constraints & Participation Constraints –**
  - **Case 2 - With Cardinality constraint and total participation from both sides –**

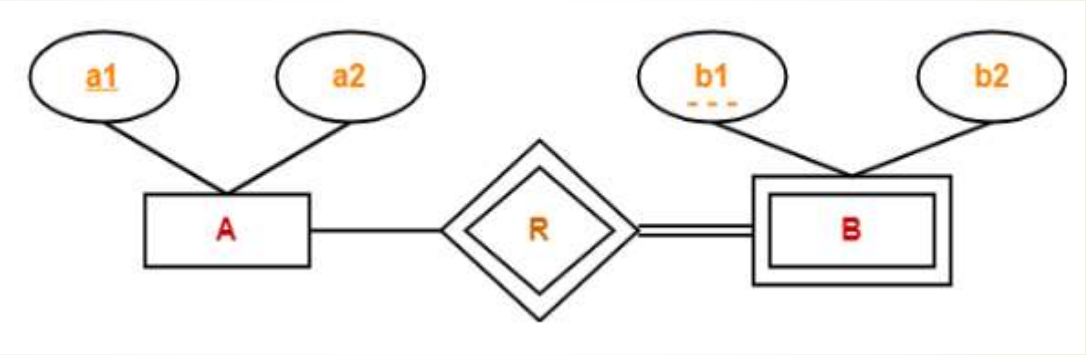


- If there is a key constraint from both the sides of an entity set with total participation, then that binary relationship is represented using only single table.
- Here, Only one table is required.
  - ARB (a1 , a2 , b1 , b2)

E-R Model

Reduction to Relational Schemas –

- **Rule 7 : For Binary Relationships with Weak Entity Set –**
  - **Weak entity set** always **appears in association with identifying relationship with total participation constraint.**



  - Here, two tables will be required-
    - A (a1 , a2)
    - BR (a1 , b1 , b2)

E-R Model

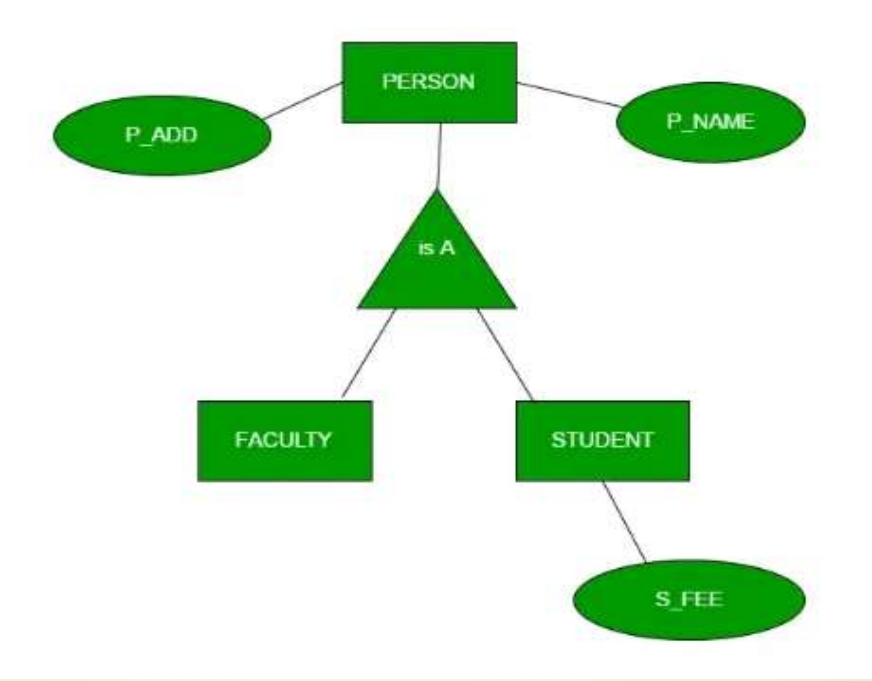
Extended E-R Features –

- Using the ER model for bigger data creates a lot of complexity while designing a database model.
- So in order to minimize the complexity Generalization, Specialization, and Aggregation were introduced in the ER model.
- These were used for data abstraction in which an abstraction mechanism is used to hide details of a set of objects.
- **Generalization –**
  - Generalization is the process of extracting common properties from a set of entities and creating a generalized entity from it.
  - It is a bottom-up approach in which two or more entities can be generalized to a higher-level entity if they have some attributes in common.
  - For Example, STUDENT and FACULTY can be generalized to a higher-level entity called PERSON as shown in Figure below

E-R Model

Extended E-R Features –

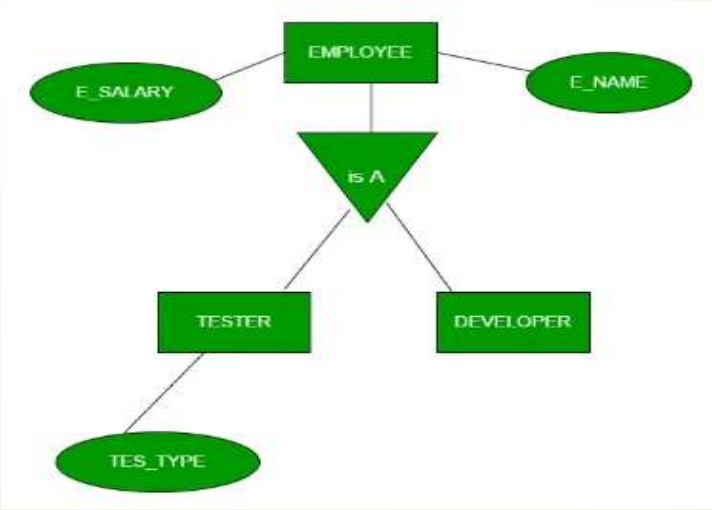
- **Generalization –**
  - In this case, common attributes like P_NAME, and P_ADD become part of a higher entity (PERSON), and specialized attributes like S_FEE become part of a specialized entity (STUDENT).



E-R Model

Extended E-R Features –

- **Specialization –**
  - In specialization, an entity is divided into sub-entities based on its characteristics.
  - It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities.
  - For Example, an EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER, etc. as shown in Figure below -



E-R Model

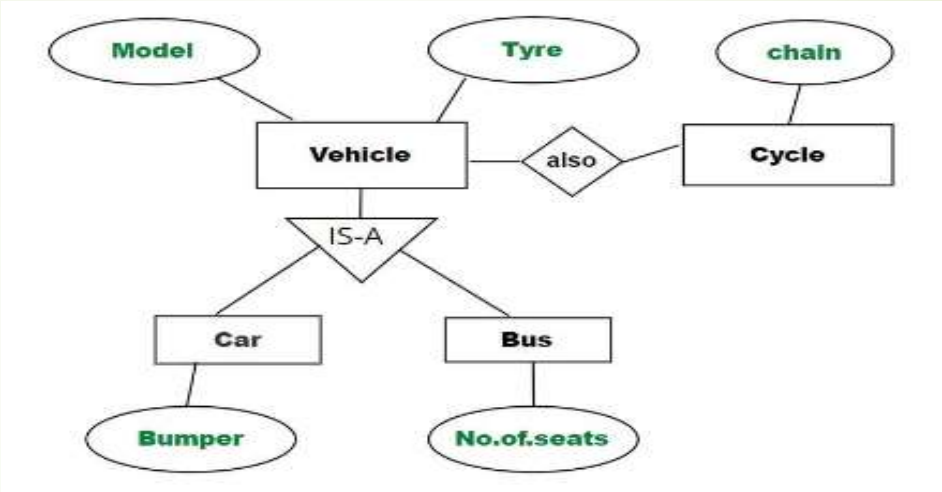
Extended E-R Features –

- **Specialization –**
  - In this case, common attributes like E_NAME, E_SAL, etc. become part of a higher entity (EMPLOYEE), and specialized attributes like TES_TYPE become part of a specialized entity (TESTER).
- *Inheritance* – It is an important feature of generalization and specialization
  - *Attribute Inheritance* –
    - It allows lower level entities to inherit the attributes of higher level entities and vice versa.
    - In below figure, **Car** entity is an inheritance of **Vehicle** entity, So Car can acquire attributes of Vehicle.
    - e.g.: car can acquire **Model** attribute of **Vehicle.**

E-R Model

Extended E-R Features –

- *Inheritance –*



- *Participation Inheritance –*
  - In this, relationships involving higher level entity set also inherited by lower level entity and vice versa.
  - In above figure, Vehicle entity has an relationship with **Cycle entity** ,So Cycle entity can acquire attributes of lower level entities i.e. Car and Bus since it is inheritance of Vehicle.

E-R Model

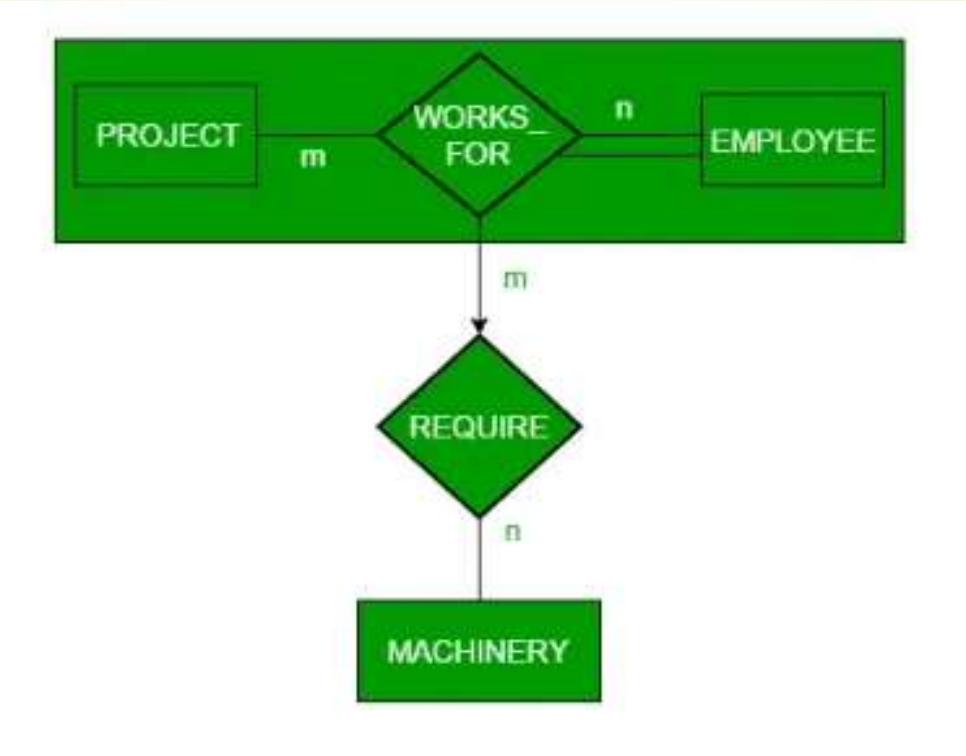
Extended E-R Features –

- **Aggregation –**
  - In some scenarios an ER diagram is not capable of representing the relationship between an entity.
  - In such a cases, a relationship with corresponding entities is aggregated into a higher-level entity.
  - Aggregation is an abstraction through which we can represent relationships as higher-level entity sets.
  - For Example, an Employee working on a project may require some machinery.
  - So, REQUIRE relationship is needed between the relationship WORKS_FOR and entity MACHINERY.
  - Using aggregation, WORKS_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into a single entity and relationship REQUIRE is created between the aggregated entity and MACHINERY.

E-R Model

Extended E-R Features –

- **Aggregation –**



E-R Model

Question Bank -

1) What is E-R model? Explain basic components of E-R model.
2) What is attribute? List & explain different types of attributes with example.
3) List & explain different types of constraints in detail with example.
4) What is key? List & explain different types of keys with example.
5) What is E-R diagram? Explain the components of E-R diagram.
6) What is mapping cardinality? How to denote mapping cardinality in E-R diagram?
7) Write a note on Weak-entity set & Strong-entity set
8) Differentiate between weak entity set and strong entity set.
9) How to reduce E-R diagrams into relational schemas?
10) Explain extended E-R features with example.